Sistema de Control de Ingreso y Salida de Equipos

Brayan Camilo Zarabanda Guerrero
Cristian Elian López Monrroy
Heider Esteban Revelo Solarte
Wilder Antonio Aristizábal Giraldo
Jhon Camilo Gonzalez Grajales

Instructor
Juan Carlos Quintero

Servicio Nacional de Aprendizaje SENA
Análisis y desarrollo de Software ADSO
Ficha 2996731
2026

Resumen

El presente manual técnico describe la arquitectura, la estructura interna, las tecnologías empleadas, los patrones de diseño adoptados y los procedimientos de instalación y configuración del Sistema de Control de Ingreso y Salida de Equipos (SCISE), desarrollado en el marco del programa de Análisis y Desarrollo de Software del Servicio Nacional de Aprendizaje (SENA), ficha 2996731. El sistema implementa una arquitectura cliente-servidor desacoplada basada en servicios REST, con un frontend construido en Astro (HTML, CSS y JavaScript), un backend desarrollado en Python con FastAPI, y una base de datos relacional PostgreSQL gestionada mediante el ORM SQLAlchemy. La autenticación se implementa mediante JSON Web Tokens (JWT) bajo un esquema de control de acceso basado en roles (RBAC). Este documento está dirigido a desarrolladores, administradores de sistemas e integradores técnicos responsables del despliegue, el mantenimiento y la evolución del sistema.

Palabras clave: arquitectura REST, FastAPI, Astro, PostgreSQL, JWT, RBAC, control de acceso, gestión de equipos.

Introducción

El Sistema de Control de Ingreso y Salida de Equipos (SCISE) surge como respuesta a la necesidad del Servicio Nacional de Aprendizaje (SENA) de contar con una plataforma digital que permita registrar, controlar y auditar de forma eficiente el movimiento de equipos tecnológicos dentro y fuera de sus instalaciones. Anteriormente, este proceso se realizaba de manera manual, lo que generaba inconsistencias en los registros, dificultades para rastrear la trazabilidad de los equipos y una carga operativa innecesaria para el personal de seguridad y administración.

SCISE propone una solución tecnológica moderna, escalable y segura, alineada con las mejores prácticas del desarrollo de software contemporáneo. El sistema se estructura sobre una arquitectura cliente-servidor completamente desacoplada, donde el frontend y el backend se comunican exclusivamente a través de una interfaz de programación de aplicaciones (API) de tipo REST. Esta separación permite que ambas capas evolucionen de forma independiente y facilita la integración futura con aplicaciones móviles u otros sistemas institucionales.

El presente manual técnico tiene como propósito documentar de manera exhaustiva todos los aspectos técnicos del sistema: su arquitectura, sus componentes, sus tecnologías, sus flujos de datos, los mecanismos de seguridad implementados, los procedimientos de instalación y los criterios para su mantenimiento y escalabilidad. Este documento no está orientado al usuario final, sino a los profesionales técnicos encargados de administrar, mantener o ampliar el sistema.

Propósito del Sistema

SCISE tiene como objetivo principal centralizar y automatizar el control de ingreso y salida de equipos tecnológicos en las instalaciones del SENA. La plataforma permite que el personal de seguridad registre en tiempo real cada movimiento de equipos, asociando cada registro a un usuario identificado mediante su carnet institucional, ya sea por ingreso manual del número de documento o mediante escaneo por código de barras con la cámara del dispositivo.

Desde una perspectiva técnica, el sistema está diseñado para cumplir con los siguientes objetivos específicos:

1. Garantizar la trazabilidad completa de cada equipo registrado en el sistema, desde su primer ingreso hasta su última salida.
2. Implementar un modelo de autenticación seguro y sin estado (stateless) basado en JWT, con control de acceso diferenciado según el rol del usuario.
3. Proveer una interfaz de programación de aplicaciones (API) RESTful documentada y extensible, que permita la integración futura con otros sistemas institucionales.
4. Ofrecer una interfaz web de alto rendimiento, accesible desde cualquier navegador moderno, sin necesidad de instalación de software adicional en el lado del cliente.
5. Diseñar una base de datos relacional íntegra, preparada para replicación y escalabilidad horizontal futura.

Alcance Técnico

El alcance técnico del sistema SCISE abarca tres grandes dominios: el frontend de usuario, el backend de servicios y la capa de persistencia de datos. A continuación, se detalla lo que está incluido dentro del alcance del presente desarrollo.

Frontend

La capa de presentación está implementada como una aplicación web estática construida con el framework Astro, que genera páginas HTML optimizadas con mínima hidratación de JavaScript en el cliente. El frontend consume los servicios del backend exclusivamente a través de peticiones HTTP/HTTPS en formato JSON. Incluye las siguientes vistas funcionales: pantalla de autenticación (login), panel principal (dashboard), módulo de registro de ingreso, módulo de registro de salida, módulo de gestión de usuarios y equipos, panel de administración con reportes y estadísticas, y módulo de configuración del sistema.

Backend

La capa de servicios está implementada en Python utilizando el framework FastAPI, ejecutado sobre el servidor ASGI Uvicorn. El backend expone una API RESTful con documentación automática generada por OpenAPI (Swagger UI). Sus responsabilidades incluyen: autenticación y autorización mediante JWT, validación estructurada de datos de entrada mediante Pydantic, lógica de negocio para el registro de movimientos, gestión de usuarios y equipos,

generación de reportes y estadísticas, y manejo estructurado de errores con códigos HTTP estándar.

Base de Datos

La capa de persistencia utiliza PostgreSQL como sistema gestor de base de datos relacional. La comunicación entre el backend y la base de datos se realiza mediante el ORM SQLAlchemy, que abstrae el motor de base de datos y permite la gestión de migraciones a través de Alembic. El modelo de datos incluye las entidades principales: usuarios, roles, equipos y movimientos (ingresos/salidas).

Fuera del Alcance

El presente desarrollo no incluye: aplicaciones móviles nativas (iOS o Android), integración con sistemas externos de biometría, módulos de facturación o inventario, ni servicios de notificaciones en tiempo real mediante WebSockets (aunque la arquitectura está preparada para ello en una iteración futura).

Arquitectura del Sistema

Patrón Arquitectónico

SCISE adopta una arquitectura cliente-servidor desacoplada, implementada bajo el patrón de arquitectura en tres capas (three-tier architecture). Este patrón organiza el sistema en tres niveles lógicos con responsabilidades claramente diferenciadas, lo que favorece el bajo acoplamiento, la alta cohesión y la facilidad de mantenimiento.

Tabla 1

Descripción de las tres capas de la arquitectura de SCISE

Capa	Tecnología	Responsabilidad principal
Presentación	Astro + HTML/CSS/JS	Interfaz de usuario, consumo de la API REST, experiencia de usuario (UX).
Servicios / Lógica	Python + FastAPI	Validaciones, autenticación, control de roles, lógica de negocio, endpoints REST.
Persistencia	PostgreSQL + SQLAlchemy	Almacenamiento relacional, integridad referencial, consultas y transacciones ACID.

Flujo de Comunicación

El flujo de comunicación del sistema sigue un esquema lineal descendente: el cliente web realiza peticiones HTTP/HTTPS en formato JSON hacia los endpoints de la API REST; el backend valida las credenciales, procesa la lógica de negocio y emite consultas al gestor de base de datos mediante el ORM SQLAlchemy; la base de datos retorna los datos requeridos, que el backend

serializa en formato JSON y devuelve al cliente. Todas las comunicaciones entre el cliente y el servidor deben realizarse sobre HTTPS en entornos de producción, garantizando la confidencialidad e integridad de los datos en tránsito.

Justificación de la Arquitectura

La adopción de una arquitectura desacoplada basada en API REST responde a múltiples criterios técnicos y estratégicos. En primer lugar, permite escalar de forma independiente el frontend y el backend según la demanda. En segundo lugar, facilita la incorporación futura de clientes alternativos, como aplicaciones móviles, sin modificar la lógica de negocio del servidor. En tercer lugar, el uso de FastAPI con soporte nativo para operaciones asíncronas (async/await) permite gestionar múltiples solicitudes concurrentes con un consumo de recursos significativamente menor en comparación con frameworks síncronos. Finalmente, PostgreSQL garantiza consistencia mediante transacciones ACID y está preparado para replicación y particionamiento horizontal en escenarios de alta concurrencia.

Tecnologías y Herramientas Utilizadas

A continuación se presenta el stack tecnológico completo del sistema SCISE, organizado por capa funcional.

Tabla 2

Stack tecnológico del sistema SCISE por capa y componente

Componente	Tecnología	Versión	Descripción
Backend	Python	3.11+	Lenguaje principal del servidor. Tipado estático opcional con mypy.
Framework API	FastAPI	0.110+	Framework ASGI moderno con validación automática por esquemas Pydantic y documentación OpenAPI.
Servidor ASGI	Uvicorn	0.29+	Servidor de alto rendimiento para aplicaciones FastAPI en producción.
ORM	SQLAlchemy	2.0+	Mapeo objeto-relacional con soporte para sesiones asíncronas.
Migraciones	Alembic	1.13+	Gestión versionada del esquema de base de datos.
Base de datos	PostgreSQL	15+	Motor relacional con soporte ACID, replicación y extensiones avanzadas.
Autenticación	JWT (PyJWT)	2.8+	Generación y validación de tokens de acceso firmados con HS256.
Hashing	bcrypt (passlib)	1.7+	Hash seguro de contraseñas con factor de costo configurable.
Frontend	Astro	4.x	Framework de generación estática con hidratación parcial en el cliente.
Estilos	CSS3	—	Estilos modulares por componente, sin dependencia de frameworks CSS externos.
Interacción cliente	JavaScript (ES2022)	—	Consumo de la API REST, manejo del DOM y lógica del escáner.

Escáner de códigos	ZXing / jsQR	—	Lectura de códigos de barras y QR mediante la cámara del dispositivo.
Proxy inverso	Nginx	1.24+	Gestión de solicitudes HTTP/HTTPS y terminación SSL en producción.
Despliegue	Linux (Ubuntu 22.04+)	—	Sistema operativo del servidor de producción.

Las versiones indicadas corresponden a las mínimas recomendadas para el entorno de producción.

Modelo de Datos

La base de datos de SCISE está modelada sobre un esquema relacional normalizado. Las entidades principales y sus relaciones se describen a continuación.

Entidades Principales

Usuarios

La entidad usuarios almacena la información de todas las personas registradas en el sistema. Cada usuario posee un identificador único, nombre completo, número de documento de identidad, correo electrónico, contraseña hasheada con bcrypt, rol asignado (administrador, seguridad, aprendiz o instructor) y estado (activo o inactivo). El campo de correo electrónico es único en la tabla y sirve como credencial de autenticación.

Equipos

La entidad equipos registra cada dispositivo tecnológico gestionado por el sistema. Sus atributos incluyen: identificador único, nombre descriptivo, número de serie (serial), tipo de equipo (portátil, tableta, proyector, etc.), descripción opcional, estado actual (disponible, en uso, inactivo) y la referencia al usuario propietario o responsable principal. El número de serie es único dentro del sistema.

Movimientos

La entidad movimientos constituye el registro histórico central del sistema. Cada registro de movimiento incluye: identificador único, tipo de movimiento (ingreso o salida), referencia al equipo involucrado, referencia al usuario responsable, y la marca de tiempo exacta del registro (timestamp). Esta entidad es inmutable: los registros no se eliminan ni modifican para garantizar la integridad de la auditoría.

Relaciones entre Entidades

Un usuario puede tener asociados múltiples equipos (relación uno a muchos entre usuarios y equipos). Cada equipo puede tener múltiples movimientos a lo largo del tiempo (relación uno a muchos entre equipos y movimientos). De igual forma, cada usuario puede ser responsable de múltiples movimientos (relación uno a muchos entre usuarios y movimientos). La integridad referencial está garantizada mediante claves foráneas con restricciones a nivel de base de datos.

Tabla 3

Entidades principales del modelo de datos de SCISE y sus atributos clave

Entidad	**Atributos principales**	**Restricciones**
usuarios	id, nombre, documento, correo, contrasena_hash, rol, estado	correo UNIQUE; documento NOT NULL; rol IN (administrador, seguridad, aprendiz, instructor)

equipos	id, nombre, serial, tipo, descripcion, estado, usuario_id	serial UNIQUE; usuario_id FK → usuarios(id)
movimientos	id, tipo_movimiento, equipo_id, usuario_id, fecha_registro	tipo IN (ingreso, salida); equipo_id FK → equipos(id); usuario_id FK → usuarios(id)

La columna contrasena_hash almacena el hash bcrypt de la contraseña original, nunca el texto plano.

Estructura del Proyecto

Estructura del Frontend

El proyecto frontend se organiza según las convenciones del framework Astro, con una separación clara entre páginas, componentes, vistas funcionales, estilos y librerías de utilidades. La estructura de directorios es la siguiente:

```
SCISE-Frontend/
├── src/
│   ├── assets/              # Recursos estáticos (imágenes, SVG)
│   ├── components/          # Componentes reutilizables
│   │   ├── navigation/      # AppHeader.astro, AppNavbar.astro
│   │   └── scanner/         # ScannerCard.astro
│   ├── features/            # Vistas por módulo funcional
│   │   ├── auth/            # LoginView.astro
│   │   ├── dashboard/       # DashboardView.astro y sub-cajas
│   │   ├── ingreso/         # IngresoView.astro y sub-cajas
│   │   ├── salida/          # SalidaView.astro y sub-cajas
│   │   ├── usuarios/        # UsuariosView.astro, modales
│   │   ├── reportes/        # ReportesView.astro y sub-cajas
│   │   └── configuracion/   # ConfiguracionView.astro y sub-cajas
│   ├── layouts/             # AppLayout, AuthLayout, BaseLayout
│   ├── lib/                 # Utilidades TS: datetime, movimientos
│   │   └── scanner/         # scanner-camera, scanner-controller
│   ├── pages/               # Rutas Astro (auth, dashboard, etc.)
│   └── styles/              # CSS modular por módulo funcional
├── astro.config.mjs         # Configuración del framework Astro
└── package.json             # Dependencias y scripts NPM
```

Estructura del Backend

El backend sigue una organización por capas funcionales, separando los modelos de datos, los esquemas de validación, los repositorios de acceso a datos, los servicios de lógica de negocio y los enrutadores de la API. La estructura recomendada es la siguiente:

```
SCISE-Backend/
├── app/
│   ├── main.py              # Punto de entrada de la aplicación
FastAPI
│   ├── core/
│   │   ├── config.py        # Variables de entorno y configuración
global
│   │   └── security.py      # Utilidades JWT, hashing y autenticación
│   ├── db/
│   │   ├── base.py          # Declarative Base de SQLAlchemy
│   │   └── session.py       # Gestión de la sesión de base de datos
│   ├── models/              # Modelos ORM (usuarios, equipos,
movimientos)
│   ├── schemas/             # Esquemas Pydantic para validación de E/S
│   ├── crud/                # Operaciones CRUD por entidad
│   ├── services/            # Lógica de negocio (movimientos,
reportes)
│   └── api/
│       └── v1/
│           └── endpoints/   # Routers: auth, users, equipment,
movements
├── alembic/                 # Migraciones de base de datos
├── requirements.txt         # Dependencias Python
└── .env                     # Variables de entorno (no versionar)
```

Descripción de la API REST

La API REST de SCISE está documentada automáticamente mediante OpenAPI. En un entorno de desarrollo, la documentación interactiva está disponible en la ruta /docs (Swagger UI) y /redoc (ReDoc). Todos los endpoints, a excepción del de autenticación, requieren un token JWT válido incluido en el encabezado de autorización HTTP con el esquema Bearer.

Prefijo Base

Todos los endpoints comparten el prefijo base /api/v1, lo que permite versionar la API y mantener compatibilidad hacia atrás en iteraciones futuras del sistema.

Módulo de Autenticación

Tabla 4

Endpoints del módulo de autenticación

Endpoint	Método	Autenticación	Descripción
/auth/login	POST	No requerida	Valida las credenciales del usuario y retorna un token JWT de acceso junto con el rol del usuario autenticado.
/auth/me	GET	Bearer JWT	Retorna la información del usuario actualmente autenticado según el token enviado.

Módulo de Usuarios

Tabla 5

Endpoints del módulo de gestión de usuarios

Endpoint	**Método**	**Roles permitidos**	**Descripción**
/users/	GET	Administrador	Lista todos los usuarios registrados con soporte de paginación, búsqueda y filtros por rol y estado.
/users/	POST	Administrador	Crea un nuevo usuario. Requiere nombre, documento, correo, contraseña y rol.
/users/{id}	GET	Administrador	Retorna los datos completos de un usuario específico por su identificador.
/users/{id}	PUT	Administrador	Actualiza los datos de un usuario existente.
/users/{id}	PATCH	Administrador	Actualiza el estado activo/inactivo de un usuario.
/users/lookup/{documento}	GET	Seguridad, Admin	Busca un usuario por número de documento. Utilizado por el módulo de escaneo.

Módulo de Equipos

Tabla 6

Endpoints del módulo de gestión de equipos

Endpoint	**Método**	**Roles permitidos**	**Descripción**
/equipment/	GET	Administrador	Lista todos los equipos con filtros por estado, tipo y paginación.

/equipment/	POST	Administrador	Registra un nuevo equipo en el sistema con su asociación a un usuario.
/equipment/{id}	PUT	Administrador	Actualiza los datos de un equipo existente.
/equipment/user/{usuario_id}	GET	Seguridad, Admin	Retorna los equipos disponibles asociados a un usuario (para el módulo de ingreso).
/equipment/active/{usuario_id}	GET	Seguridad, Admin	Retorna los equipos con ingreso activo de un usuario (para el módulo de salida).

Módulo de Movimientos

Tabla 7

Endpoints del módulo de registro de movimientos

Endpoint	**Método**	**Roles permitidos**	**Descripción**
/movements/ingreso	POST	Seguridad, Admin	Registra el ingreso de uno o más equipos asociados a un usuario. Acepta un arreglo de identificadores de equipos.
/movements/salida	POST	Seguridad, Admin	Registra la salida de uno o más equipos con ingreso activo asociados a un usuario.
/movements/history	GET	Administrador	Retorna el historial completo de movimientos con filtros por fecha, tipo, usuario y equipo, con paginación.
/movements/summary	GET	Administrador	Retorna las estadísticas generales: total de equipos, equipos dentro y fuera, y movimientos del día.

Seguridad y Autenticación

Modelo de Autenticación JWT

El sistema implementa autenticación basada en JSON Web Tokens (JWT) bajo una arquitectura stateless. Al autenticarse exitosamente, el backend genera un token firmado digitalmente con el algoritmo HMAC-SHA256 (HS256), utilizando una clave secreta almacenada como variable de entorno. El token incluye en su payload el identificador del usuario, su rol y la fecha de expiración. El cliente almacena el token en el almacenamiento local del navegador (localStorage) y lo incluye en el encabezado Authorization: Bearer <token> en cada solicitud subsecuente.

El servidor valida el token en cada solicitud protegida mediante una dependencia de FastAPI (Depends). Si el token ha expirado, está malformado o ha sido manipulado, el servidor retorna un código HTTP 401 (Unauthorized). No existe mecanismo de refresh token en la versión actual; cuando el token expira, el usuario debe autenticarse nuevamente.

Control de Acceso Basado en Roles (RBAC)

El sistema define tres niveles de acceso diferenciados mediante el modelo RBAC. Cada endpoint del backend verifica el rol del usuario autenticado antes de ejecutar la operación solicitada. Un usuario que intente acceder a un recurso para el que no tiene permisos recibirá un código HTTP 403 (Forbidden).

Tabla 8

Matriz de permisos por rol en el sistema SCISE

Funcionalidad	Administrador	Seguridad	Aprendiz	Instructor
Registrar ingreso de equipos	Sí	Sí	No	No
Registrar salida de equipos	Sí	Sí	No	No
Consultar historial de movimientos	Sí	No	No	No
Ver estadísticas del panel	Sí	No	No	No
Gestionar usuarios (CRUD)	Sí	No	No	No
Gestionar equipos (CRUD)	Sí	No	No	No
Acceder a configuración del sistema	Sí	No	No	No
Consultar equipos por documento	Sí	Sí	No	No

Los roles Aprendiz e Instructor son usados como identidades para la asociación de equipos y movimientos, pero no tienen acceso operativo al sistema.

Almacenamiento Seguro de Contraseñas

Las contraseñas de los usuarios nunca se almacenan en texto plano en la base de datos. El sistema utiliza la biblioteca passlib con el algoritmo bcrypt para generar un hash seguro con un factor de costo configurable (por defecto, 12 rondas de hashing). Cuando un usuario intenta autenticarse, el sistema compara el hash almacenado con el hash generado a partir de la contraseña ingresada, sin exponer nunca el valor original.

Medidas Adicionales de Seguridad

El sistema implementa las siguientes medidas complementarias de seguridad: todas las comunicaciones entre el cliente y el servidor deben realizarse sobre HTTPS en producción, con un certificado SSL gestionado por Nginx; todas las entradas de datos son validadas estructuralmente mediante esquemas Pydantic antes de ser procesadas, lo que previene inyecciones SQL y ataques de tipo deserialización insegura; el ORM SQLAlchemy ejecuta consultas parametrizadas, eliminando el riesgo de inyección SQL directa; y los permisos son verificados a nivel de cada endpoint del backend, independientemente de lo que el frontend muestre u oculte al usuario.

Gestión de Errores

El sistema implementa un manejo centralizado y estructurado de errores en el backend. Todas las excepciones son capturadas por manejadores de errores registrados en la aplicación FastAPI, que retornan respuestas en formato JSON con la siguiente estructura uniforme:

```
{
  "detail": "Mensaje descriptivo del error",
  "status_code": 422
}
```

Los códigos de estado HTTP utilizados por el sistema y su significado en el contexto de SCISE se detallan a continuación:

Tabla 9

Códigos de estado HTTP utilizados por la API de SCISE

Código HTTP	**Nombre**	**Contexto de uso en SCISE**
200 OK	Éxito	La solicitud fue procesada correctamente y se retorna el recurso o la confirmación de la operación.
201 Created	Creado	Se creó exitosamente un nuevo recurso (usuario, equipo o movimiento).
400 Bad Request	Solicitud inválida	Los datos enviados no cumplen con el esquema esperado o contienen valores no permitidos.
401 Unauthorized	No autenticado	El token JWT está ausente, ha expirado o es inválido.
403 Forbidden	Sin permisos	El usuario está autenticado pero no tiene el rol necesario para la operación solicitada.
404 Not Found	No encontrado	El recurso solicitado (usuario, equipo o movimiento) no existe en la base de datos.

409 Conflict	Conflicto	Se intenta crear un recurso con un valor único duplicado (por ejemplo, correo o serial ya registrado).
422 Unprocessable Entity	Error de validación	Pydantic rechazó los datos de entrada por no cumplir las reglas de validación del esquema.
500 Internal Server Error	Error del servidor	Ocurrió un error inesperado en el servidor. Se registra en los logs para diagnóstico.

Instalación y Configuración

Requisitos del Entorno de Producción

El servidor de producción debe cumplir con los siguientes requisitos mínimos para el correcto funcionamiento del sistema SCISE:

Tabla 10

Requisitos mínimos del servidor de producción

Componente	Especificación mínima
Sistema operativo	Ubuntu Server 22.04 LTS o superior
Procesador	2 núcleos virtuales (vCPU) a 2.0 GHz o superior
Memoria RAM	4 GB (se recomiendan 8 GB para entornos con alta concurrencia)
Almacenamiento	20 GB SSD (espacio inicial; crecerá según el volumen de movimientos registrados)
Python	3.11 o superior
Node.js	18 LTS o superior (requerido para la compilación del frontend Astro)
PostgreSQL	15 o superior
Nginx	1.24 o superior
Certificado SSL	Let's Encrypt u otro proveedor válido para HTTPS

Instalación del Backend

Los siguientes pasos describen el proceso de instalación del backend en el servidor de producción. Se asume que Python 3.11+ y PostgreSQL ya están instalados en el servidor.

Paso 1. Clonar el repositorio y crear el entorno virtual

```
git clone https://github.com/sena-adso/scise-backend.git
cd scise-backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Paso 2. Configurar las variables de entorno

Cree un archivo .env en la raíz del proyecto backend con las siguientes variables de entorno. Este archivo no debe ser versionado en el repositorio de código fuente.

```
DATABASE_URL=postgresql+asyncpg://usuario:contrasena@localhost:5432/sci
se_db
SECRET_KEY=clave_secreta_larga_y_aleatoria_minimo_32_caracteres
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=480
ALLOWED_ORIGINS=https://dominio-frontend.sena.edu.co
```

Paso 3. Crear la base de datos y ejecutar las migraciones

```
createdb -U postgres scise_db
alembic upgrade head
```

Paso 4. Ejecutar el servidor con Uvicorn

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

Para un entorno de producción, se recomienda gestionar el proceso con systemd o supervisord para garantizar el reinicio automático ante fallos.

Compilación e Instalación del Frontend

Paso 1. Instalar dependencias y configurar la variable de entorno

```
cd scise-frontend
npm install
echo 'PUBLIC_API_BASE_URL=https://api.dominio.sena.edu.co/api/v1' >
.env
```

Paso 2. Compilar el proyecto para producción

```
npm run build
```

El comando anterior genera una carpeta dist/ con los archivos estáticos optimizados del frontend. Esta carpeta debe ser servida por Nginx.

Configuración de Nginx

Se debe configurar Nginx como proxy inverso para el backend y como servidor de archivos estáticos para el frontend. La configuración básica es la siguiente:

```
server {
    listen 443 ssl;
    server_name dominio.sena.edu.co;

    # Frontend
```

```
    location / {
        root /var/www/scise/dist;
        try_files $uri $uri/ /index.html;
    }

    # Backend API
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Escalabilidad y Mantenimiento

Estrategia de Escalabilidad

La arquitectura desacoplada de SCISE permite escalar cada capa de forma independiente. El backend puede escalarse horizontalmente desplegando múltiples instancias de Uvicorn detrás de un balanceador de carga (como Nginx o HAProxy), dado que la autenticación JWT es stateless y no requiere sesiones compartidas entre instancias. El frontend, al ser un conjunto de archivos estáticos, puede distribuirse mediante una red de entrega de contenido (CDN) para reducir la latencia y mejorar la disponibilidad global. La base de datos PostgreSQL puede configurarse con réplicas de lectura para distribuir la carga de consultas, reservando la instancia principal exclusivamente para operaciones de escritura.

Estrategia de Mantenimiento

El uso de migraciones versionadas con Alembic garantiza que cualquier cambio en el esquema de la base de datos pueda aplicarse y revertirse de forma controlada. Cada nueva funcionalidad o corrección debe ir acompañada de su migración correspondiente. Se recomienda establecer una política de backups automáticos de la base de datos con una frecuencia mínima diaria, utilizando pg_dump con retención de al menos 30 días. Las actualizaciones de dependencias deben evaluarse y probarse en un entorno de staging antes de ser desplegadas en producción.

Variables de Entorno y Secretos

Todos los secretos del sistema (clave JWT, credenciales de base de datos, etc.) deben gestionarse exclusivamente mediante variables de entorno y nunca deben estar presentes en el código fuente ni en el repositorio de versiones. Se recomienda el uso de un gestor de secretos como HashiCorp Vault o AWS Secrets Manager en entornos con mayores requisitos de seguridad.

Restricciones Técnicas

El sistema SCISE opera bajo las siguientes restricciones técnicas, derivadas de los documentos de alcance y requerimientos del proyecto:

6. Compatibilidad de navegador. El sistema es compatible con las versiones actuales de Google Chrome, Mozilla Firefox y Microsoft Edge. No se garantiza compatibilidad con Internet Explorer ni con versiones de navegadores con más de dos años de antigüedad.
7. Función de escaneo por cámara. La lectura de códigos de barras mediante la cámara del dispositivo requiere que el navegador tenga permisos para acceder a la cámara y que la página sea servida bajo HTTPS o desde localhost. En entornos sin cámara disponible, el ingreso de documentos debe realizarse de forma manual.
8. Dependencia de conectividad. El sistema requiere conexión activa a Internet o a la red interna institucional para funcionar. No existe una modalidad offline ni sincronización diferida de datos.
9. Inmutabilidad de los registros de movimientos. Los registros de ingreso y salida no pueden ser editados ni eliminados una vez creados, para preservar la integridad del historial de auditoría.
10. Autenticación de un solo factor. La versión actual del sistema implementa autenticación mediante contraseña únicamente. No se incluye autenticación de dos factores (2FA) en el alcance de este desarrollo.